FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
DEPARTAMENTUL DE INFORMATICĂ
SPECIALIZAREA CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

PROIECT CALCUL NUMERIC

APLICAȚIE DESCOMPUNEREA VALORILOR SINGULARE
SISTEM DE RECOMANDARE A ȚINUTELOR

Proiect realizat de:
Chelan Maria-Estera
Neculae Mara-Daniela
Sturzoiu Maria-Filofteia

BUCUREȘTI
2026

Cuprins

1 Introducere **2**
1.1 Motivație 2
1.2 Descrierea aplicației alese 2

2 Descompunerea după Valori Singulare **3**
2.1 Descrierea matematică a metodei 3
2.1.1 Bidiagonalizarea Householder 3
2.1.2 Formarea matricii tridiagonale simetrice 3
2.1.3 Iterația QR 3
2.1.4 Extragerea valorilor singulare și ordonarea lor 4
2.2 Utilizarea metodei în aplicația selectată 4
2.2.1 Identificarea culorii 4
2.2.2 Formarea recomandărilor 5
2.3 Comparația funcțiilor definite cu cele din librăria NumPy 5

3 Interpretarea rezultatelor **7**
3.1 Grafice relevante 7

4 Concluzii **8**

5 Bibliografie **10**

1 Introducere

1.1 Motivație

În contextul dinamic al vieții contemporane, oamenii se confruntă adesea cu problema formării unei ținute pentru o zi importantă, aglomerată sau imprevizibilă, mai ales atunci când vremea este schimbătoare. Persoanele cu dificultăți în a lua decizii sau lipsă de creativitate pot întâmpina situații neplăcute în încercarea de a alege ce să poarte la un eveniment sau la o activitate nouă, pierzând timp prețios.

Observând impactul practic al acestei probleme, proiectul nostru propune dezvoltarea unei aplicații capabile să automatizeze și să eficientizeze procesul de selecție vestimentară prin implementarea unui sistem de recomandare avansat. Fundația matematică a acestei soluții se bazează pe algoritmul de Descompunere după Valori Singulare. În analiza numerică și *data science*, SVD este recunoscută ca o metodă fundamentală pentru reducerea dimensionalității datelor și extragerea caracteristicilor latente, având aplicabilitate extinsă în arii precum compresia de imagini, filtrarea zgomotului din semnale și sistemele de filtrare colaborativă.

Alegerea acestei teme integrează astfel rigoarea algebrei liniare numerice cu o aplicație interactivă. Prin utilizarea proprietăților geometrice și structurale ale componentelor obținute din SVD, proiectul demonstrează modul în care concepte matematice abstracte pot fi transpuse în soluții algoritmice eficiente pentru optimizarea deciziilor din viața cotidiană.

1.2 Descrierea aplicației alese

Aplicația pe care am dezvoltat-o utilizează o bază de date formată din articole vestimentare încărcate de utilizator și date curente cu privire la vreme și stilul necesar. Cu ajutorul funcțiilor de rezolvare prin metoda SVD, aplicația prelucrează informațiile furnizate cu scopul de a obține o vestimentație optimă care să îndeplinească cerințele și să se asorteze după culoare.

Baza de date conține informații despre articole, ce au următoarele caracteristici: ID numărul de ordine, $Imagine$ imaginea asociată cu fundal alb, $Categorie$ (poate fi: „Top" pentru tricouri, bluze și cămăși, „Bottom" pentru fuste și pantaloni, „Extra" pentru geci și sacouri și „Pantofi"), $Anotimp$, $Stil$ (poate fi: „Casual", „Formal", „Elegant" sau „Sportiv") și codul RGB de culoare, completat de aplicația noastră.

În codul dezvoltat, aceste informații sunt atribuite unor matrici care sunt prelucrate prin metoda de Descompunere după Valorile Singulare, iar soluțiile obținute sunt afișate.

2 Descompunerea după Valori Singulare

2.1 Descrierea matematică a metodei

Descompunerea după Valori Singulare este o metodă de factorizare în algebra liniară, ce descompune o matrice în alte trei matrici, oferind o cale de reprezentare a datelor în funcție de valorile lor singulare.

În algoritmul pe care l-am proiectat, am ales să utilizăm o bidiagonalizare, urmată de factorizarea QR, inspirată din metoda Golub-Kahan. Scopul acesteia este de a obține descompunerea matricii A sub forma:

$$A = U\Sigma V^T \tag{1}$$

Unde U și V sunt matrici ortogonale, iar Σ conține valorile singulare pe diagonala principală.

Pentru a ajunge la forma finală, parcurgem etapele următoare:

2.1.1 Bidiagonalizarea Householder

Din dorința de a aduce matricea A la o formă mai simplă, o reducem la o matrice bidiagonală B, dreptunghiulară, care are elemente doar pe diagonala principală și pe cea de deasupra ei. Astfel, se aplică transformări ortogonale de reflexie la stânga și la dreapta matricii A:

$$B = U_{house}^T A V_{house} \tag{2}$$

O matrice Householder are forma:

$$H = I - 2\frac{vv^T}{v^T v} \tag{3}$$

Aceasta anulează la stânga elementele de sub diagonala principală dintr-o coloană, iar la dreapta, elementele de la dreapta supradiagonalei dintr-o linie.

Matricea rezultat B are forma următoare:

$$B = \begin{pmatrix} d_1 & e_1 & 0 & \dots & 0 & 0 \\ 0 & d_2 & e_2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & d_{n-1} & e_{n-1} & \\ 0 & 0 & \dots & 0 & d_n & \end{pmatrix} \tag{4}$$

2.1.2 Formarea matricii tridiagonale simetrice

Înmulțind matricea B cu transpusa ei, obținem o matrice tridiagonală simetrică M. Astfel, valorile proprii ale lui M sunt pătratele valorilor singulare ale lui B, implicit, ale lui A.

2.1.3 Iterația QR

Pentru a găsi valorile proprii ale matricii simetrice M, am folosit Algoritmul QR prin metoda Gram-Schmidt. La fiecare iterație k se descompune matricea curentă

$$T_k = Q_k R_k \tag{5}$$

Unde Q_k este ortogonală, iar R_k este superior triunghiulară. Apoi, se reînmulțesc invers:

$$T_k = R_k Q_k \tag{6}$$

Această transformare este o asemănare ortogonală:

$$T_k + 1 = Q_k^T T_k Q_k \tag{7}$$

Prin repetare, elementele din afara diagonalei tind la 0, iar matricea T converge către o matrice diagonală. Elementele de pe diagonala sa sunt valorile proprii λ_i.

Matricea V_{qr} acumulează toate transformările $V_{qr} = Q_1 Q_2 \dots Q_k$ reprezentând vectorii proprii ai lui $B^T B$

2.1.4 Extragerea valorilor singulare și ordonarea lor

În această etapă finală, algoritmul colectează toate părțile calculate anterior, le sortează conform convențiilor matematice și reconstruiește cele trei matrici finale: $U\Sigma V^T$. Extragem diagonala din T, aplicăm radicalul și obținem valorile singulare σ.

Prin sortare, algoritmul află ordinea de la cea mai mare valoare la cea mai mică și rearanjează atât valorile singulare, cât și vectorii din matricea V pentru a respecta standardul matematic.

Pentru construcția matricii Σ, se creează o matrice goală de dimensiunea inițială și se plasează pe diagonala ei valorile singulare sortate.

În calculul matricii U, pentru vectorii care corespund unor valori singulare nenule, aceasta este calculată direct prin formula derivată din definiția metodei SVD.

$$U_i = \frac{AV_i}{\sigma_i} \tag{8}$$

Cum matricea A are mai puține valori singulare decât numărul de linii, restul vectorilor din U sunt calculați folosind procesul de ortogonalizare Gram-Schmidt, asigurându-ne că U devine o matrice ortogonală completă.

În final, funcția returnează tripletul complet U, Σ, V^T, care îndeplinește condiția $A = U\Sigma V^T$.

2.2 Utilizarea metodei în aplicația selectată

Metoda de descompunere după valorile singulare apare în aplicația proiectată în două părți fundamentale: în identificarea culorii articolului vestimentar din imagine și în fomarea recomandărilor finale.

2.2.1 Identificarea culorii

Pentru a identifica culoarea unui articol dintr-o imagine, folosind metoda SVD, se creează hărți de contrast care separă obiectul principal de fundal. Odată ce SVD izoleaza obiectul de interes, algoritmii matematici extrag precis valorea RGB, fără a fi încurcați de umbre, reflexii de lumină sau fundal.

Codul separă imaginea redimensionată la dimensiunile de 40x40 în trei matrici bidimensionale R, G și B, iar pentru fiecare canal rulează algoritmul SVD prin bidiagonalizare. Astfel, imaginea este reconstruită folosind primele două valori singulare ce captează cea mai mare parte din variația de lumină din imagine și următoarea cea mai importantă variație, cum ar fi tranzițiile geometrice importante dintre obiect și fundal. Adunând aceste componente pentru fiecare canal, obținem harta de contrast.

În continuare, codul utilizează harta pentru a curăța datele. Pentru filtrarea după constrast se păstrează doar pixelii unde contrastul este mai mare de 20% din maxim, eliminând fundalul plat. În filtrarea după distanța față de colțuri, se calculează culoarea medie a colțurilor și se elimină pixelii care seamănă cu ea. Pentru eliminarea reflexiilor, se calculează al 85-lea procent al luminozității și se elimină pixelii prea luminoși. Pentru pixelii rămași, se aplică mediana. Astfel, se extrage nuanța cea mai stabilă. În Figura 1 se poate observa rezultatul obținut pentru primul articol ales.

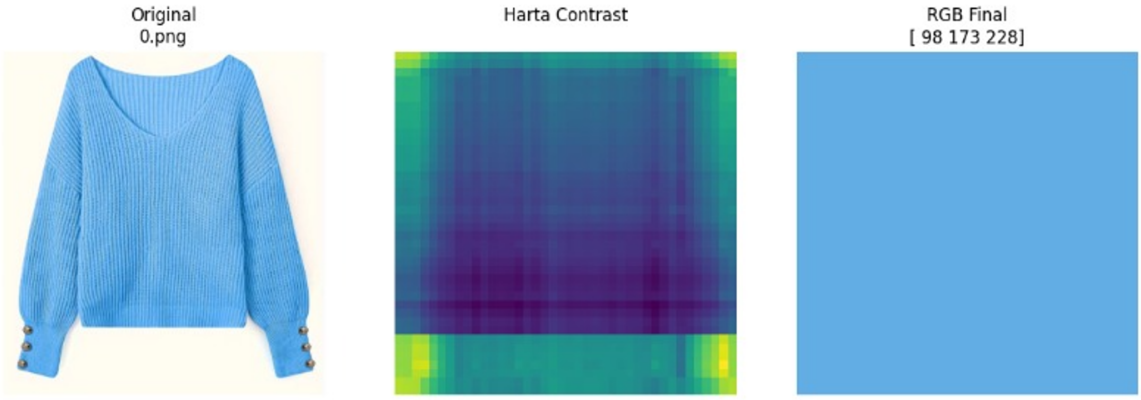


Figura 1: *Primul articol, harta sa de contrast și culoarea extrasă*

2.2.2 Formarea recomandărilor

Pornind de la matricea A și vectorul y formați în urma prelucrării bazei de date, dorim să obținem vectorul de ponderi w pentru a determina preferințele utilizatorului. Cum matricea A nu este pătratică, sistemul este supradeterminat, deci alegem să calculăm pseudoinversa A^+

Prin metoda SVD, descompunem matricea A în $A = USV^T$, iar inversarea acestei matrici devine simplă și stabilă numeric.

$$A^+ = VS^{-1}U^T \tag{9}$$

Înmulțind pseudoinversa obținută cu vectorul y, obținem vectorul w, iar prin înmulțirea acestuia cu matricea A, se generează un vector de scoruri prezise pentru toate hainele din garderobă.

În final, algoritmul selectează piesele pentru ținută după culoare. În cazul unei egalități, codul recurge la folosirea scorurilor calculate drept factor decisiv.

2.3 Comparația funcțiilor definite cu cele din librăria NumPy

Funcțiile definite diferă de cele din librăria NumPy prin diverse aspecte ce includ metoda de calcul, performanța și viteza, stabilitatea numerică și precizia.

Din punctul de vedere al metodei de calcul, funcția $SVD_Bidiagonal$ definită în cod, folosește reflexii Householder pentru a aduce matricea la o formă bidiagonală, calculează matricea M și aplică iterații QR pentru a găsi vectorii și valorile proprii, extrage valorile singulare prin radical și reconstruiește manual matricea U. Spre deosebire de aceasta, funcția predefinită din NumPy nu folosește iterații QR clasice pe matricea M, fiindcă se pierde precizia numerică în urma înmulțirii matricii B cu transpusa ei. De asemenea, funcția folosește librării avansate și utilizează algoritmul de tip Divide et Impera direct pe matricea bidiagonală.

Din perspectiva performanței și a vitezei, funcția construită funcționează foarte lent pentru matrici cu dimensiuni mari, în timp ce funcția predefinită poate gestiona cu ușurință matrici voluminoase.

Precizia numerică a funcției dezvoltate în cod este afectată de utilizarea algoritmului Gram-Schmidt care este sensibil la erorile de rotunjire ale calculatoarelor, iar vectorii U și V își pot pierde orgonalitatea perfectă. Funcția definită în NumPy are o precizie aproape de limita hardware-ului, garantând că matricile rămân ortogonale cu o eroare infimă.

Așadar, funcția definită în NumPy funcționează la o scară mai largă pentru date complexe într-un mod mai eficient și precis decât funcția construită în cod. Cu toate acestea, rezultatele obținute sunt aceleași, evidențiind utilitatea metodei Descompunerii după Valorile Singulare indiferent de metoda de calculare a acesteia.

3 Interpretarea rezultatelor

Pentru rularea algoritmului final, utilizatorul va introduce datele cu privire la vremea de afară și stilul dorit. În urma rulării, se va afișa ținuta formată din articolele corespunzătoare alese din baza de date. În Figura 2, se poate observa soluția oferită pentru cerința unei vestimentații la 0 grade, stilul Elegant.



Figura 2: *Exemplu*

3.1 Grafice relevante

Figura 3 ilustrează grafic procesul de convergență al algoritmului de calcul SVD. Pentru primele 20 de iterații se poate observa cum eroarea scade brusc, dar are și mici oscilații. Acest fenomen este cauzat de utilizarea algoritmului QR clasic. Astfel, deși matricea converge global către forma diagonală, eroarea maximă poate prezenta mici oscilații locale și creșteri temporare înainte de a începe să scadă constant. Pentru iterațiile 20-35, algoritmul a găsit direcția corectă, iar elementele scad rapid, eroarea scăzând de la 0.1 la sub 0.01. După iterația 35, curba devine o linie aproape dreaptă care coboară constant, indicând o convergență liniară stabilă. La 100 de iterații, eroarea absolută maximă ajunge la aproximativ 0.0003.

Din convergența algoritmului deducem faptul că acesta funcționează corect, fără erori matematice grave. În dezavantaj, rularea algoritmului de la 60 la 100 de iterații aduce un plus de precizie, dar consumă timp de calcul suplimentar pentru un câștig vizual minor în aplicație.

Graficul din Figura 4 prezintă proiecția a celor 49 de articole de îmbrăcăminte din matricea de input A, în spațiul latent bidimensional obținut în urma rulării algoritmului SVD. Se poate observa cu ușurință proprietatea SVD-ului de a grupa automat obiectele similare: hainele de vară/casual s-au poziționat în cadranul superior, în timp ce piesele elegante s-au grupat în partea opusă. Acest comportament geometric certifică corectitudinea reducerii dimensionalității, permițând funcției de potrivire să calculeze corect distanțele pentru recomandarea outfiturilor.

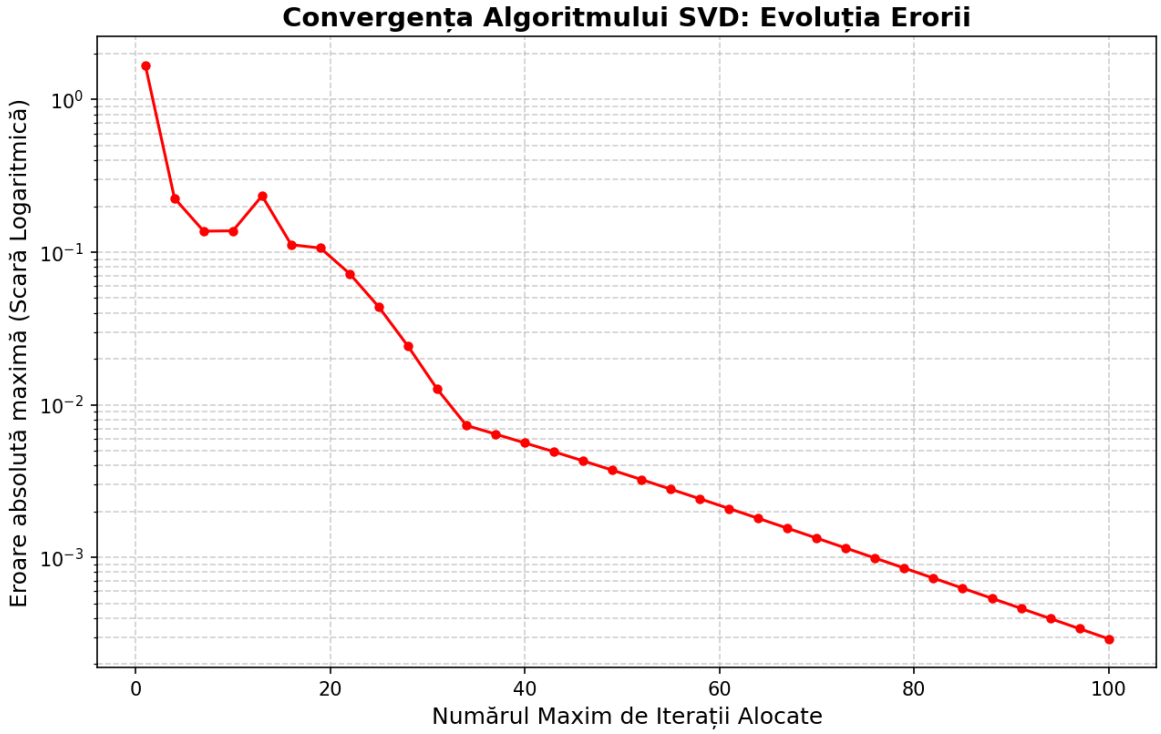


Figura 3: *Graficul de evoluție a erorii*

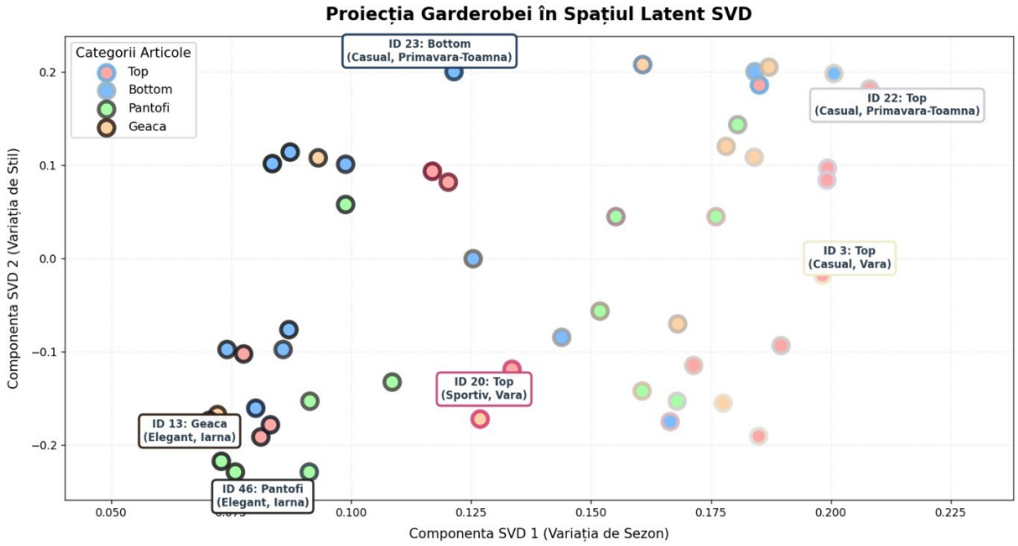


Figura 4: *Garderoba în spațiul latent*

4 Concluzii

În urma realizării acestui proiect, am constatat utilitatea metodei Descompunerii după Valorile Singulare în numeroase aplicații practice, împreună cu avantajele și dezavantajele implementării acesteia. Implementarea acestui algoritm ne-a permis nu doar să ne însușim cunoștințe avansate de algebră liniară și programare în limbajul Python, dar ne-a oferit și o perspectivă clară asupra compromisurilor dintre o implementare algoritmică fundamentală și utilizarea librăriilor industriale optimizate.

Proiectul a demonstrat versatilitatea SVD prin abordarea a două direcții complet diferite, dar complementare, în ingineria software: procesarea de imagini și extragerea de caracteristici.

De asemenea, un punct central al proiectului l-a reprezentat implementarea algoritmului propriu bazat pe reflexii Householder și iterații QR. Comparația acestuia cu funcția predefinită din NumPy a demonstrat ineficiența pe seturi mari de date, însă rezultatele identice obținute prin ambele metode au validat corectitudinea algoritmului.

În același timp, proiectarea acestei aplicații a reprezentat o adevărată oportunitate de a ne dezvolta abilitățile de comunicare, lucru în echipă și de adaptare la medii noi de lucru și de învățare. Repartizarea sarcinilor, corelarea modulelor de cod și depanarea colectivă a erorilor de sincronizare ne-au oferit o experiență asemănătoare unui flux de lucru real din industria IT.

Dincolo de aspectele tehnice ale ingineriei software și ale matematicii, alegerea acestei teme a avut la bază o puternică motivație subiectivă. Fiind toate trei pasionate de lumea modei, dezvoltarea acestui proiect a reprezentat o activitate extrem de plăcută și stimulantă, oferindu-ne șansa de a îmbina rigoarea științifică cu un interes personal comun. Această sinergie ne-a determint să privim dincolo de cerințele academice actuale și ne-a determinat să ne propunem extinderea proiectului într-o aplicație comercială completă, intitulată „TryOn”. Ne dorim ca pe viitor să integrăm în cadrul acesteia module avansate de realitate augmentată (AR) pentru probarea virtuală a hainelor, transformând astfel acest algoritm de recomandare într-un produs software interactiv și inovator.

În concluzie, realizarea acestui proiect ne-a permis să înțelegem modul în care concepte teoretice, adesea percepute ca abstracte, pot fi transpuse în aplicații practice, evidențiind totodată rolul esențial al matematicii în dezvoltarea și optimizarea algoritmilor utilizați.

5 Bibliografie

Golub G, Van Loan C. Matrix Computations; 1989.

The Golub-Kahan Method of Computing a Singular Value Decomposition - Evelien Bogers